



Lista 5 — TypeScript e Padrões Avançados

Aulas 41 a 50

Desenvolvimento Web

Exercícios sobre TypeScript, testes automatizados, boas práticas e padrões de projeto. Cenários de código escalável e manutenível em equipe.

Exercício 1

Cenário: Equipe de 5 devs precisa refatorar um projeto React para TypeScript.

1. Converta um componente simples de JavaScript para TS: defina interface Props { nome: string; idade?: number; onClick: () => void; }.
2. Explique os tipos: any, unknown, never, void. Em que situações cada um é apropriado?
3. Use Generics: crie uma função fetchData<T>(url: string): Promise<T> que retorna dados tipados.
4. Configure tsconfig.json no projeto: strict: true, noImplicitAny, strictNullChecks. Quais erros surgem e como corrigi-los?

Exercício 2

Cenário: Garantir qualidade do código com testes automatizados antes do deploy.

1. Instale Vitest + Testing Library (@testing-library/react). Escreva teste para um componente Contador: renderiza, clica no botão e verifica se o valor incrementa.
2. Teste um hook customizado useCounter: verifique se increment() e decrement() funcionam, se valor nunca fica negativo.
3. Implemente teste de integração: formulário de login, preenche campos, clica em "Entrar", verifica se navegou para /dashboard.
4. Configure GitHub Actions para rodar os testes a cada push e não permitir merge se os testes falharem.

Exercício 3

Cenário: Refatorar código legado com padrões de projeto (Design Patterns).

1. Identifique no código um "god component" que acumula muitas responsabilidades. Aplique SRP (Single Responsibility) quebrando em componentes menores.
2. Implemente o padrão Observer para um sistema de notificações: subject (GerenciadorNotificacoes) notifica observers (EmailNotifier, SMSNotifier, PushNotifier).
3. Use o padrão Factory para criar diferentes tipos de usuário (Admin, Editor, Viewer) com permissões distintas.
4. Pesquise sobre Compound Components no React: como o padrão `<Select><Option>` facilita APIs declarativas? Implemente um `<Tabs>` com `<Tab>` e `<TabPanel>`.

Exercício 4

Cenário: App precisa funcionar offline (PWA) com Service Worker e cache.

1. Registre um Service Worker que intercepta fetch requests e serve do cache (Cache First para assets, Network First para API).
2. Crie um manifesto.json com: nome, ícones 192x192 e 512x512, tema color, display standalone.
3. Implemente uma página "offline" customizada que é exibida quando o usuário está sem internet.
4. Teste no Chrome DevTools > Application > Service Workers: simule offline, verifique o cache, atualize o SW.

Requisitos

Node.js 18+, Vite + React + TypeScript, Vitest. `npm create vite@latest -- --template react-ts`. GitHub para Actions.